

GameObject handler 인터페이스 설계

목적

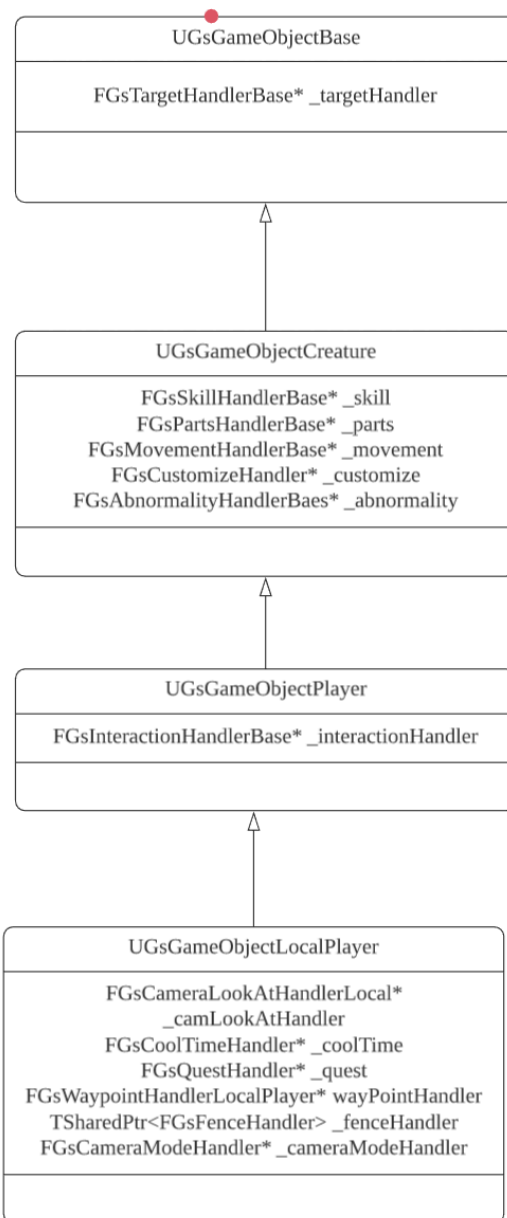
1. GameObject내에 Component 위상인 Handler 객체를 인터페이스하여 생성, 소멸, Tick 등의 통합관리한다.
2. 구현된 Handler 클래스의 위상 및 실효성을 재검토한다.

1. 사례분석(로컬 플레이어 기준으로 조사)

- 핸들러 분포조사

GameObject Handler 사례조사

태현 백 | July 20, 2020



추가할 인터페이스

IGsGameObjectHandlerBase

```
virtual void Initialize(UGsGameObjectBase* owner) = 0;
virtual void Finalize() = 0
virtual void Update(float delta) = 0
```

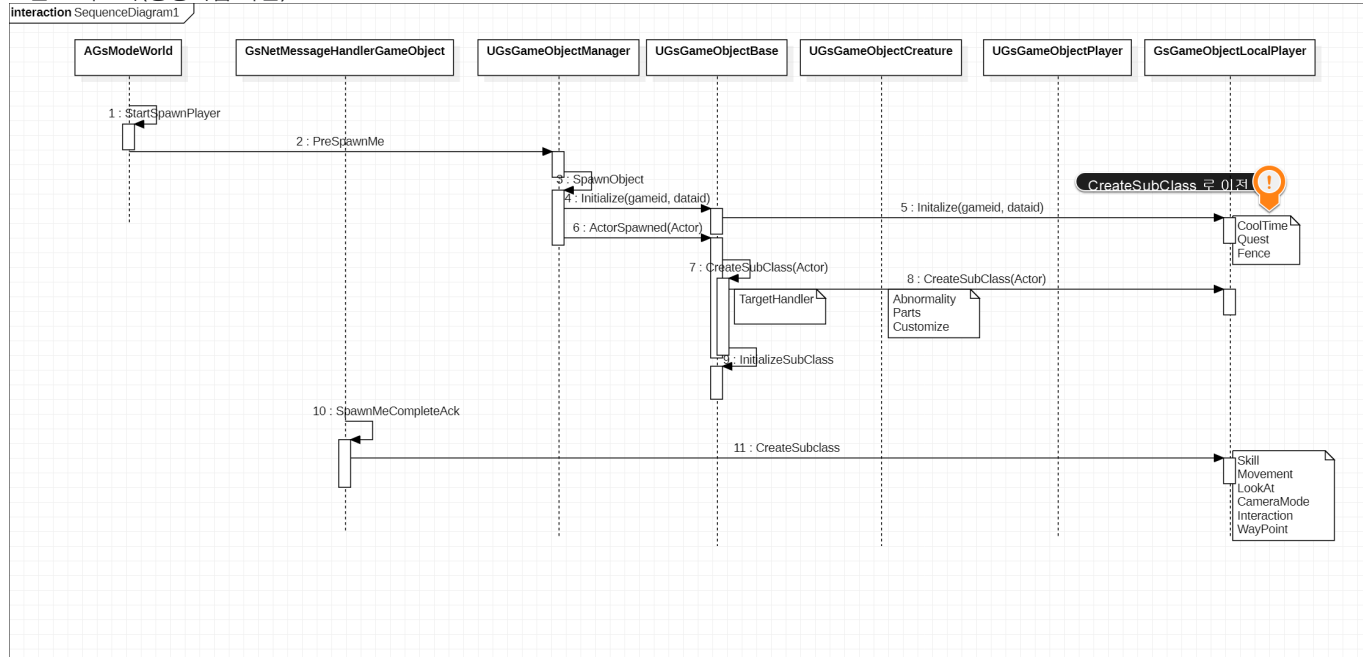
추가고려사항

1. 핸들러 Get / Set

- Getter 만 공개
- 빠른 형변환 지원 필요

- 인터페이스를 통한 Array 관리처리 (다만 Getter 처리에 형변환 Cost를 지불할 필요있는지 확인해보자)
- 변수명 규칙 / Ptr 타입 통일화 필요
- FGsWaypointHandlerLocalPlayer ⇒ 목적과 위상이 핸들러에 부합되지 않음 → 담당자 피드백 : FGsMovementHandler내에 관련 기능을 구현한다.

• 호출 스택조사(생성시점 확인)



◦ CreateSubClass(Actor*) 함수내에서 핸들러 클래스를 생성하는 것이 의도였으나 현재는 2가지 이유로 파편화되어 의도를 상실

1. Initialize(gameid, dataid) 호출시점에 생성 : 단순 함수초기화 위치라 판단하고 추가됨 => 강한 구조 드리븐 필요
2. 인자없는 11번 CreateSubclass 분기(외부호출 타이밍) : 캐릭터 생성이 2Step으로 분기되면서 목적성을 가지고 분기 => 목적성이 들어나지 않고 동일 이름이 존재하여 복잡성 증대

2.처리 내역(r19986)

주요 변경사항 정리

1. GameObject내에 핸들러는 IGsGameObjectHandler 인터페이스 상속을 통해 처리 및 관리
2. 생성 및 초기화가 동시에 진행됨 (순서가 매우 중요함)

생성 및 초기화

```

void AppendInitialize(IGsGameObjectHandler* inHandler)
{
    check(inHandler);
    inHandler->Initialize(this);
    _handlerHolder.Add(inHandler);
}
  
```

3. Geter는 형변환을 수반한다.

Getter

```

template <typename T = IGsGameObjectHandler>
T* GetGameObjectHandler(GameObjectHandlerType inType)
{
    IGsGameObjectHandler* foundhandler = nullptr;
    for (IGsGameObjectHandler* handler : _handlerHolder)
    {
        if (handler && handler->IsA(inType))
        {
            foundhandler = handler;
            break;
        }
    }
    return foundhandler == nullptr ? nullptr : FGsClassUtil::FCheckStaticCast<T>(foundhandler);
}
  
```

4. 핸들러 관리는 GameObjectBase에서..(Update, Finalize) 에서, 생성 및 초기화는 각각의 상속된 클래스(드리븐)에서..

잔여이슈 정리

- Interface처리는 추가되었지만, 기존 코드(방대한) 내용때문에 기존 Getter 코드들은 유지
 - 파생이슈 1: 형변환 관련 코드처리로 상수성관련 코드들 변경 됨 → Const 상실 : **이범용** 친천히 감싼함수를 제거
- 리프 크리처(로컬유저, 몬스터, 타유저)는 2원화된 초기화를 사용한다.
 - CreateSubClass(AActor* Owner) → 타유저, 몬스터
 - CreateSubclass() → 여기에서 로컬에만 해당하는 핸들러들 생성 → 이유는 결국 각 핸들러의 역할은 최종생성된 후에 동작해야하는데, 로컬플레이어는 2원화된 생성로직 (1. 퍼시에서 내캐릭터가 위치한 지형을 로딩 → 2. 서버에 지형로딩 완료 → 3. 내캐릭 생성) 1, 3번 이유

